BAC Blanc

ÉPREUVE D'ENSEIGNEMENT DE SPÉCIALITÉ NUMÉRIQUE ET SCIENCES INFORMATIQUES

Mercredi 8 avril 2026

Durée de l'épreuve : 3 heures 30

L'usage de la calculatrice n'est pas autorisée.

Le candidat traite les trois exercices proposés.

Ce sujet comporte 8 pages numérotées de 1/8 à 8/8.

Exercice 1 (5 points)

Un palindrome est un mot qui se lit de la même manière de la gauche vers la droite que de la droite vers la gauche (exemple : « kayak » est un palindrome).
On propose ci-dessous une fonction pour tester si un mot est un palindrome.

```
def tester_palindrome(chaine):
        if len(chaine) < 2:
                return True
        elif chaine[0] != chaine[-1]:
                return False
        else:
                chaine = chaine[1:-1]
                return tester_palindrome(chaine)
```

1. On saisit, dans la console, l'instruction suivante : `tester_palindrome('kayak')`
   Combien de fois est appelée la fonction `tester_palindrome` lors de l'exécution de cette instruction ? On veillera à compter l'appel initial.

   > Pour le 1er appel on a chaine = 'kayak', pour le 2e appel chaine = 'aya', pour le 3e appel chaine = 'y'. On a donc 3 appels.

2. (a) Justifier que la fonction `tester_palindrome` est récursive. soit la chaîne de caractères sur laquelle elle s'applique.

   > La fonction tester_palindrome s'appelle elle-même, elle est donc récursive.

   (b) Expliquer pourquoi l'appel à la fonction `tester_palindrome` se terminera quelle que

   > À chaque appel récursif, la chaine de caractère perd 2 caractères (le premier et le dernier). Après un certain nombre d'appels, il restera donc moins de 2 caractères. Si la chaine de caractères contient moins de 2 caractères, on tombe alors dans le cas de base (len(chaine) ¡ 2), ce qui provoque l'arrêt des appels récursifs.

3. La saisie, dans la console, de l'instruction `tester_palindrome(53235)` génère une erreur.
   (a) Parmi les quatre propositions suivantes, indiquer le type d'erreur affiché :

- ZeroDivisionError
- ValueError
- **TypeError**
- IndexError

(b) Proposer sur la copie une ou plusieurs instructions qu'on pourrait écrire entre la ligne 1 et la ligne 2 du code de la fonction `tester_palindrome` et permettant d'afficher clairement cette erreur à l'utilisateur.

```
assert type (chaine) is str, "parametre␣de␣type␣str␣attendu"
```

4. Écrire le code d'une fonction itérative (non récursive) `est_palindrome` qui prend en paramètre une chaîne de caractères et renvoie un booléen égal à `True` si la chaîne de caractères est un palindrome, `False` sinon.

```
def est_palindrome(chaine) :
  i = 0
  j = len(chaine) - 1
  while i < j:
    if chaine[i] != chaine[j]:
      return False
    i = i + 1
    j = j - 1
  return True
```

Exercice 2 (10 points)

Les vacances d'été se rapprochent et le propriétaire d'une pension pour animaux gère les places dont il dispose à l'aide d'une base de données dont voici le schéma relationnel :

- `client(num_client, nom_client, prenom_client, mail_client, tel_client)`
- `animal(num_animal, nom_animal, categorie_animal, taille_animal, num_client)`
- `cage(num_cage, taille_cage, secteur_cage)`
- `reservation(num_reservation, date_debut_reservation, date_fin_reservation, num_client, num_animal, num_cage)`

Ci-dessous, on donne des extraits des tables `client`, `animal`, `cage` et `reservation`.

Extrait de la table `client` :

num_client	nom_client	prenom_client	mail_client	tel_client
16	Dupont	Marc	marc.dupont@mail.com	0604050401
345	Morel	Fabien	fabien.morel@mail.com	0700051020

Extrait de la table `animal` :

num_animal	nom_animal	categorie_animal	taille_animal	num_client
22	Yuki	souris	petit	16
112	Balou	chat	moyen	141
320	Api	chien	grand	237
423	Rex	chien	moyen	259
491	Rex	chien	petit	345

Extrait de la table `cage` :

num_cage	taille_cage	secteur_cage
4	grand	chien
12	petit	chien
23	moyen	chien
31	moyen	chien
32	petit	rongeur
33	grand	chat

Extrait de la table `reservation` :

num_reservation	date_debut_reservation	date_fin_reservation	num_client	num_animal	num_cage
44	2022-08-23	2022-08-25	26	12	12
45	2022-07-11	2022-07-22	345	491	23
46	2022-08-11	2022-08-22	345	491	23
47	2022-08-23	2022-09-10	345	491	23
48	2022-10-11	2022-10-22	345	491	23

1. **Étude du schéma relationnel**

   (a) Pour chaque attribut de la relation `cage`, spécifier son type, en utilisant le tableau des types suivant :

`CHAR(t)` `VARCHAR(t)`	Texte de longueur fixe de t caractères. Texte de longueur variable de t caractères au maximum.
`INT`	Nombre entier de -2^{31} à $2^{31}-1$ (signé) ou de 0 à $2^{32}-1$ (non signé).
`FLOAT`	Réel à virgule flottante.
`DATE` `DATETIME`	Date format AAAA-MM-JJ. Date et heure format AAAA-MM-JJ HH:MI:SS.

num_cage INT
taille_cage CHAR(5)
secteur_cage VARCHAR(7)

   (b) Préciser, pour la relation `reservation`, le nom de la clé primaire pouvant être utilisée.

clé primaire table reservation : num_reservation

   (c) Indiquer, pour la relation `reservation`, la ou les clés étrangères (ou secondaires) et en indiquer l'utilité.

clés étrangères table reservation :
num_client (permet de faire le lien avec la table client),
num_animal (permet de faire le lien avec la table animal),
num_cage (permet de faire le lien avec la table cage)

2. **Requêtes SQL**

(a) Indiquer le résultat de l'exécution de la requête suivante :

```
SELECT nom_animal
FROM animal
WHERE categorie_animal = 'chien';
```

Api
Rex
Rex

(b) Indiquer le résultat de l'exécution de la requête suivante :

```
SELECT DISTINCT categorie_animal
FROM animal;
```

Souris
Chat
Chien

(c) Écrire une requête SQL permettant d'afficher le nom du chien du client 'Marc Dupont'.

```
SELECT nom_animal
FROM animal
WHERE num_client = 16
```

(d) Écrire une requête SQL permettant d'afficher les noms de tous les clients dont l'animal a occupé la cage numéro 23.

```
SELECT client.nom_client
FROM client
JOIN reservation ON reservation.num_client = client.num_client
WHERE reservation.num_cage = 23
```

(e) Un nouvel animal doit être enregistré dans la base de données qui contient actuellement 491 animaux. Il s'appelle Suki, c'est un chat de petite taille dont le propriétaire a déjà été enregistré sous le numéro 342.
Écrire la requête SQL permettant d'insérer ces nouvelles données dans la base de données.

```
INSERT INTO animal
VALUES
(492, "Suki", "chat", "petit", 342)
```

3. **Programmation Python**
Suite à une panne, le responsable de la pension n'a plus accès à sa base de données. Heureusement, il avait fait une sauvegarde de ses tables au format csv. Il les a importées à l'aide d'un programme Python, chacune sous la forme d'une liste de dictionnaires.

Pour simplifier, on considérera que la table reservation est la liste de dictionnaires suivante :

```
reservation = [
{'num_reservation' : 44, 'date_debut_reservation' : '2022-08-23',
'date_fin_reservation' : '2022-08-25', 'num_client' : 26,
'num_animal' : 12, 'num_cage' : 12},
{'num_reservation' : 45, 'date_debut_reservation' : '2022-07-11',
'date_fin_reservation' : '2022-07-22', 'num_client' : 345,
'num_animal' : 491, 'num_cage' : 23},
{'num_reservation' : 46, 'date_debut_reservation' : '2022-08-11',
'date_fin_reservation' : '2022-08-22', 'num_client' : 345,
'num_animal' : 491, 'num_cage' : 23},
{'num_reservation' : 47, 'date_debut_reservation' : '2022-08-23',
'date_fin_reservation' : '2022-09-10', 'num_client' : 345,
'num_animal' : 491, 'num_cage' : 23},
{'num_reservation' : 48, 'date_debut_reservation' : '2022-10-11',
'date_fin_reservation' : '2022-10-22', 'num_client' : 345,
'num_animal' : 491, 'num_cage' : 23}]
```

(a) On donne ci-dessous, le code Python d'une fonction `mystere`.

```
def mystere(table, date):
        liste= []
        for ligne in table:
                if ligne['date_debut_reservation'] == date:
                        liste.append(ligne['num_client'])
        return liste
```

Indiquer l'affichage produit par l'exécution de la ligne de code suivante :
`print(mystere(reservation, '2022-08-23')`

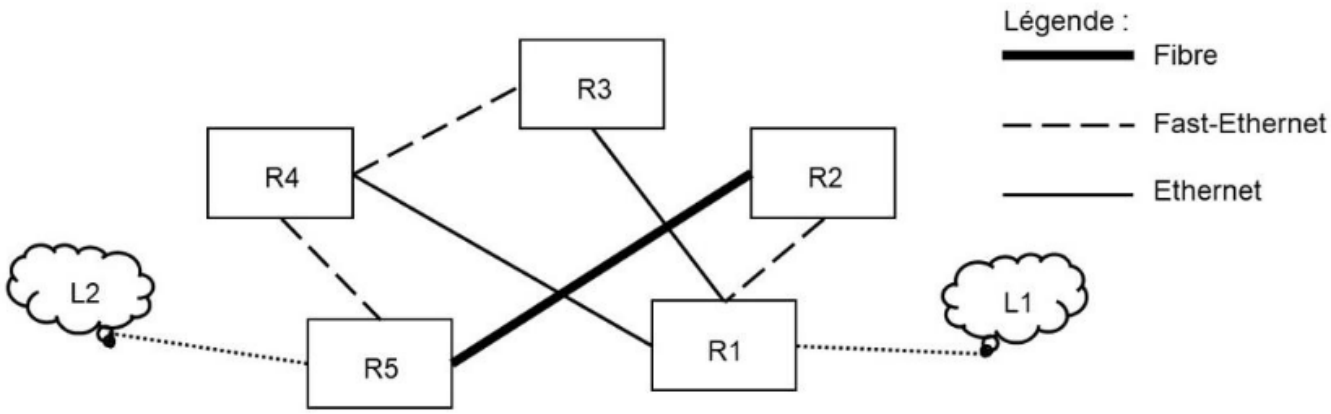
```
affichage : [26, 345]
```

(b) Le responsable de la pension veut obtenir le nombre de réservations qui ont été effectuées pour un numéro de client donné.
Ecrire le code de la fonction `nombre_reservation` afin de respecter la spécification donnée ci-dessous.

```python
def nombre_reservation(table,numero_client):
 """ parametres :
  table : liste de dictionnaires representant les reservations;
  numero_client : nombre entier representant le numero du client concerne;

  Valeur renvoyee : un nombre entier donnant le nombre d'occurences du
     numero du client concerne.
 """
  compteur = 0
  for d in table:
    if d['num_client'] == numero_client:
      compteur = compteur + 1
  return compteur
```

4. **Protocole OSPF**
La sauvegarde de la base de données est stockée sur le réseau local L1, relié au routeur R1 du réseau suivant.



L'ordinateur de bureau du responsable de la pension fait partie du réseau local L2.
Les réseaux locaux L1 et L2 font partie d'un réseau constitué de 5 routeurs (R1, R2, R3, R4, R5), de liaisons de communication dont les bandes passantes sont de 1 Gbit/s pour la Fibre, 100 Mbit/s pour Fast-Ethernet et 10 Mbit/s pour Ethernet.

On s'intéresse ici au protocole de routage OSPF. Le protocole OSPF cherche à minimiser la somme des coûts des liaisons entre les routeurs empruntés par un paquet de données.
Le coût C d'une liaison est donné par :

$$C = \frac{10^8}{d}$$

où d est la bande passante en bit/s de la liaison.

(a) Pour passer de L1 à L2, le chemin R1 - R2 - R5 utilisant la fibre a le coût le plus faible. Calculer ce coût.

Fibre = 0,1 ; Fast-Ethernet = 1 ; Ethernet = 10
1 + 0,1 = 1,1

(b) La liaison entre R2 et R5 a été coupée en raison de travaux. Déterminer la route permettant de relier le réseau L1 au réseau L2 selon le protocole OSPF. Justifier.

R1 -¿ R4 -¿ R5 pour un coût égal à 10+1 = 11

(c) Nous considérons maintenant que la liaison entre R2 et R5 est rétablie.
Sur votre copie, reproduire et compléter la table de routage du routeur R3 selon le protocole OSPF.

Destination	**Passerelle**	**Métrique**
R1		
R2		
R3		
R4		
R5		

Table de routage du routeur R3

(d) Expliquer pourquoi cette table serait différente si l'on utilisait le protocole RIP.

> La métrique utilisée dans le protcole RIP est différente. On compte le nombre de routeur traversé. La passerelle peut donc être différente et la métrique également.
> Par exemple, pour la destination R2, la passerelle serait R1 et la métrique 2.

Exercice 3 (5 points)

Dans cet exercice, on s'intéresse à la notation polonaise inversée (NPI) d'une expression mathématique. Dans cette notation, l'opérateur est placé après les nombres sur lesquels il s'applique. On se limitera aux expressions faisant intervenir des nombres entiers et les quatre opérateurs : $+$, $-$, $\times$, $/$
Par exemple, l'expression $4 \times (5+7)$ s'écrit, en NPI : $4\ 5\ 7\ +\ \times$

L'évaluation de l'expression $12/2-4+5\times 3$, écrite en NPI : $12\ 2\ /\ 4-5\ 3\times +$ est détaillée ci-dessous. Cette expression s'évalue à 17 de la manière suivante :

- lorsqu'on rencontre un premier opérateur $(+,\ -,\ /,\ \times)$, on évalue l'opération avec les deux nombres situés juste avant cet opérateur :

$$\underbrace{12\ 2\ /}_{12/2=6}\ 4-5\ 3\times +$$

- on remplace cette opération par le résultat :

$$6\ \ 4-5\ 3\times +$$

- on continue la lecture de l'expression :

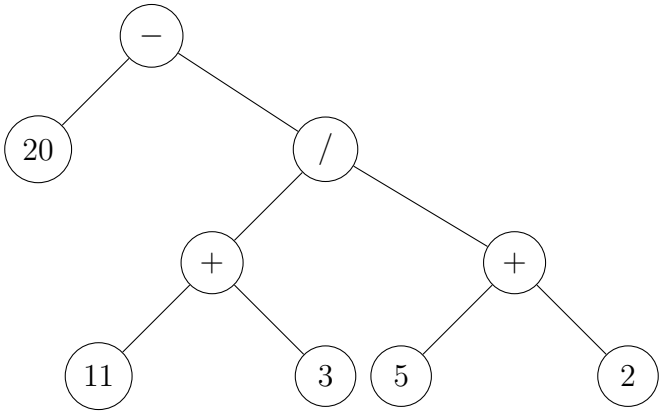
$$\underbrace{6\ 4\ -}_{6-4=2}\ 5\ 3\times + \text{ devient } 2\ \ \underbrace{5\ 3\times}_{5\times 3=15}\ + \text{ devient } 2\ 15\ +\ \text{ qui vaut } 17$$

1. Donner la valeur de l'expression suivante écrite en NPI :

$$15\ 5-4\ 12\ +\ \times$$

> (15 - 5) x (4 + 12) = 160

2. On donne l'arbre binaire suivant :



Indiquer, parmi les quatre différents parcours d'arbre ci-dessous celui qui permet d'obtenir la succession des symboles correspondant à la notation NPI suivante :

$$20\ 11\ 3 + 5\ 2 +\ /\ -$$

- un parcours en largeur ;
- un parcours en profondeur préfixe ;
- un parcours en profondeur infixe ;

> - un parcours en profondeur postfixe (ou suffixe).

3. On utilisera une liste de symboles pour écrire la notation polonaise inversée d'une expression mathématique. Par exemple, $6\ 2\ 3 +\ \times\ 94\ 1 - +$ sera représentée par la liste en langage python : `[6, 2, 3, +, *, 94, 1, -, +]`

On considère à présent l'algorithme ci-dessous, écrit en pseudo-code, qui reçoit une liste de symboles correspondant à la notation polonaise inversée d'une expression et renvoie l'arbre binaire associé. **Algorithme `créer_arbre(liste_symboles)`**

```
P ← créer une pile vide
pour chaque élément de liste_symboles :
    si élément est un entier :
        a ← construire l'arbre de racine élément et n'ayant pas de sous-arbre droit
            ni gauche
    sinon : # élément est le symbole d'un opérateur
        a_droit ← dépiler P
        a_gauche ← dépiler P
        a ← construire l'arbre de racine élément,
            ayant pour sous-arbre gauche a_gauche et pour sous-arbre droit a_droit
    empiler a dans P.
a ← dépiler P
renvoyer a
```

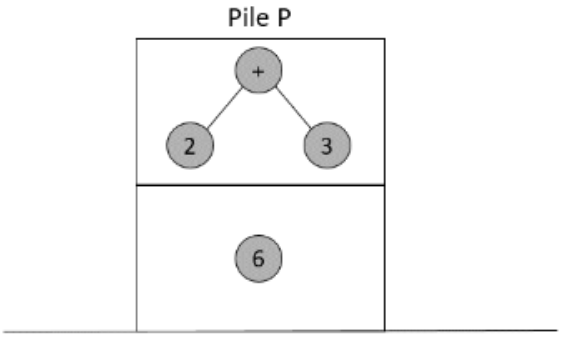
L'algorithme précédent nécessite l'utilisation d'une pile, dont les éléments sont des arbres. On précise que :

- l'action empiler `a` dans `P` insère l'élément `a` en haut de la pile `P` ;
- l'action dépiler `P` renvoie la valeur de l'élément en haut de la pile `P` et le supprime de la pile.

(a) Des deux acronymes LIFO et FIFO, indiquer lequel permet de décrire, de manière générale, la structure de pile. Donner la signification des quatre lettres qui composent cet acronyme.

LIFO

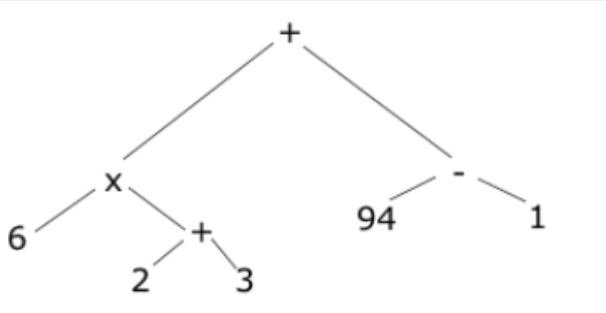
(b) On applique l'algorithme précédent sur la liste `[6, 2, 3, +, *, 94, 1, -, +]`.
À la fin de l'exécution du quatrième tour de la boucle pour, l'état de la pile est le suivant :



Indiquer le nombre d'élément(s) contenu(s) dans la pile `P` et dessiner l'état de la pile, à la fin de l'exécution du cinquième tour de la boucle pour.

1 élément

(c) Dessiner l'arbre binaire représentant l'expression mathématique dont la notation polonaise inversée est la liste `[6, 2, 3, +, *, 94, 1, -, +]`.



4. On souhaite écrire une fonction en langage Python qui permet d'évaluer une expression mathématique écrite avec la notation polonaise inversée NPI.
On suppose qu'une telle expression est représentée par un arbre binaire obtenu à l'aide de l'algorithme précédent. On dispose pour cela de quatre fonctions :
   - `est_vide(arb)` qui renvoie `True` si l'arbre binaire `arb` est vide, `False` sinon ;
   - `racine(arb)` qui renvoie la valeur de la racine de l'arbre binaire `arb` ;
   - `gauche(arb)` qui renvoie le sous-arbre gauche de l'arbre binaire `arb` ;
   - `droit(arb)` qui renvoie le sous-arbre droit de l'arbre binaire `arb`.

La fonction `evaluer` donnée ci-dessous est écrite en Python.
Cette fonction prend en paramètre un arbre binaire `arb` représentant une expression mathématique écrite en NPI et renvoie la valeur de cette expression.

```
def evaluer(arb):
   if est_vide(gauche(arb)) and instruction 1:
      res = instruction 2
   elif instruction 2 == '+':
      res = evaluer(instruction 3) + instruction 4
   elif instruction 2 == '-':
      res = evaluer(instruction 3) - instruction 4
   elif instruction 2 == '*':
      res = evaluer(instruction 3) * instruction 4
   else:
      res = evaluer(instruction 3) / instruction 4
   return res
```

Quatre instructions seulement permettent de compléter ce code : **instruction 1**, **instruction 2**, **instruction 3** et **instruction 4**.
Écrire sur la copie le code de chacune de ces quatre instructions.

```
instruction 1 : est_vide(droit(arb)):
instruction 2 : racine(arb)
instruction 3 : evaluer(gauche(arb))
instruction 4 : evaluer(droit(arb))
```